Analysis Results

bankparserapi-main.zip

Report Date
2026-04-22 12:16:52
Report Author
dctuser
Classification Method
By severity
Product Version
10.16
Rules Version
10.127506

Confidentiality Note

This report is intended only for the person(s) or entity to which it is addressed and contains confidential and privileged information. If you are not the intended recipient, you must not view, use, copy, disclose, or otherwise disseminate this report or any part of it. Doing so is strictly prohibited, and may result in legal proceedings. If you received this in error, please notify the sender immediately and destroy any copies of this information.

1 Project Information 4

Dynamics by vulnerabilities 5

Scan History 7

2 Scan Information 58/58 2026-04-22 11:54:49 14

Scan Statistics 14

Language Statistics 15

Vulnerability List 17

Analysis Results 19

WAF Setup Instructions 32

Scan Settings 34

3 Export Settings 36

01 Project Information

Project Name

bankparserapi-main.zip

UUID

4112c9b1-5763-473f-af2f-e635755a41a6

Project in DerScanner

Dynamics by vulnerabilities

Vulnerabilities are divided by severity level: critical, medium, low and info.

CRITICAL LEVEL

Likely to lead to compromise confidential data and violation of the integrity of the system.

MEDIUM LEVEL

May be less likely to lead to compromising confidential data and violating the integrity of the system, or are less serious security

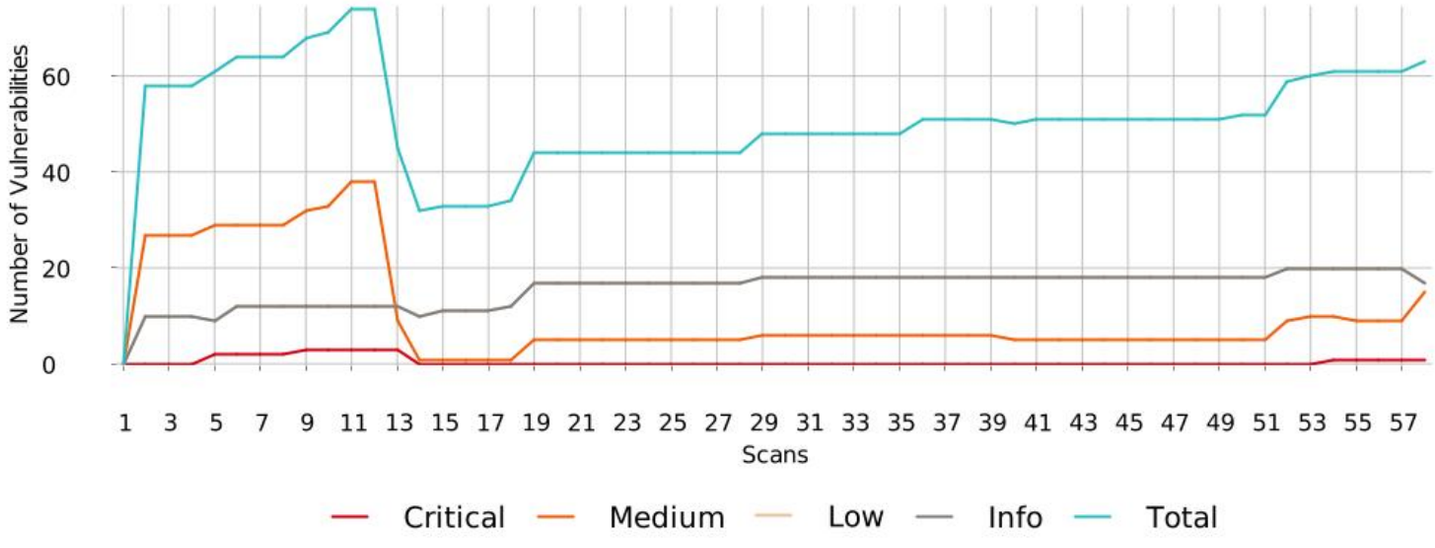
LOW LEVEL

Can become a potential security risk.

INFORMATION

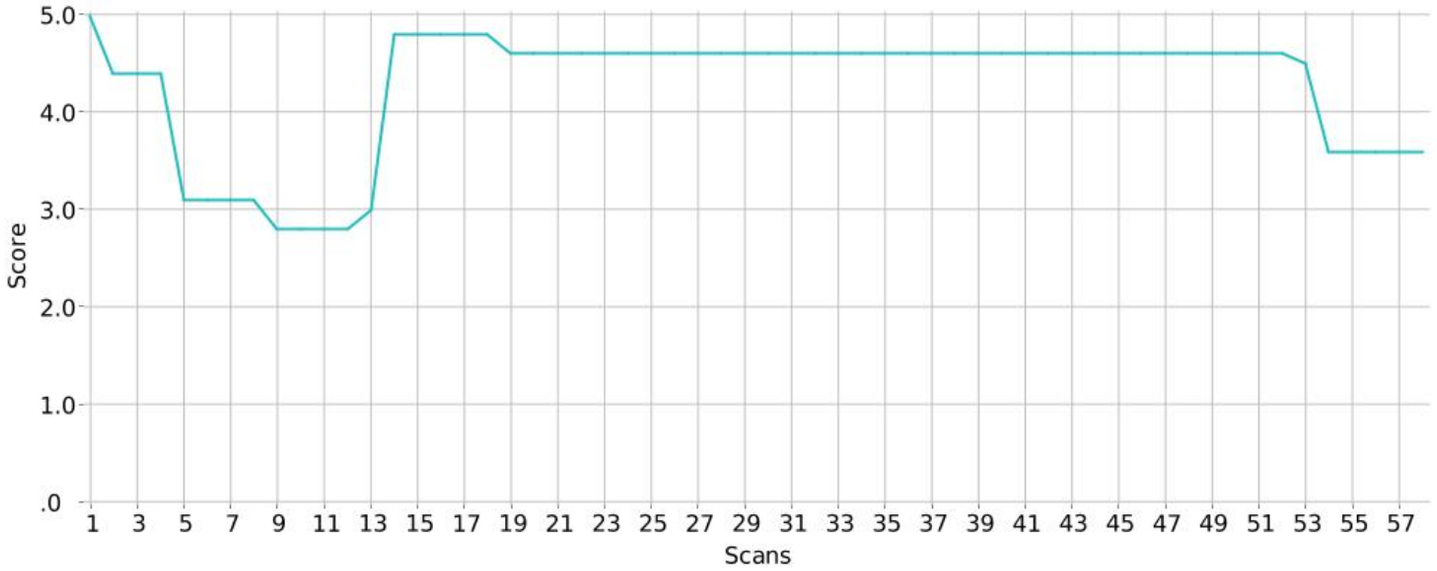
Signal a violation of good programming practice.

First of all, pay attention to vulnerabilities of critical and medium levels.



Dynamics by rating

The app score is calculated on a scale from 0 to 5. Score is calculated based on the number of critical and medium level vulnerabilities. The impact of critical vulnerabilities is greater than that of medium level vulnerabilities, and does not take into account the amount of code. Medium level vulnerabilities are taken into account based on their frequency and total number of source code lines.



Scan History

	Date and Time	Status	Languages	Lines of Code	Number of Vulnerabilities					Score
					Critical	Medium	Low	Info	Total	
58/58	2026-04-22 11:54:49	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	45 587	1	15	17	30	63	3.6/5.0
57/58	2026-04-22 08:21:18	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	45 377	1	9	20	31	61	3.6/5.0
56/58	2026-04-22 07:37:18	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	45 225	1	9	20	31	61	3.6/5.0
55/58	2026-04-22 07:00:49	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	45 191	1	9	20	31	61	3.6/5.0
54/58	2026-04-22 06:09:00	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	45 164	1	10	20	30	61	3.6/5.0
53/58	2026-04-21 11:53:57	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	43 706	0	10	20	30	60	4.5/5.0
52/58	2026-04-20 18:40:42	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	43 528	0	9	20	30	59	4.6/5.0
51/58	2026-04-20 09:46:20	completed	Config files, Python, JavaScript,	41 619	0	5	18	29	52	4.6/5.0

	Date and Time	Status	Languages	Lines of Code	Number of Vulnerabilities					Score
					Critical	Medium	Low	Info	Total	
			PL/SQL, T-SQL, HTML5							
50/58	2026-04-19 14:21:49	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	41 522	0	5	18	29	52	4.6/5.0
49/58	2026-04-15 16:59:51	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 928	0	5	18	28	51	4.6/5.0
48/58	2026-04-15 10:52:23	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 929	0	5	18	28	51	4.6/5.0
47/58	2026-04-14 05:48:50	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 876	0	5	18	28	51	4.6/5.0
46/58	2026-04-14 05:28:58	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 875	0	5	18	28	51	4.6/5.0
45/58	2026-04-13 17:51:48	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 875	0	5	18	28	51	4.6/5.0
44/58	2026-04-13 17:40:58	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 875	0	5	18	28	51	4.6/5.0
43/58	2026-04-11 11:42:13	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 767	0	5	18	28	51	4.6/5.0
42/58	2026-04-08 06:26:58	completed	Config files, Python,	39 743	0	5	18	28	51	4.6/5.0

	Date and Time	Status	Languages	Lines of Code	Number of Vulnerabilities					Score
					Critical	Medium	Low	Info	Total	
			JavaScript, PL/SQL, T-SQL, HTML5							
41/58	2026-04-03 13:04:21	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	39 155	0	5	18	28	51	4.6/5.0
40/58	2026-04-03 05:57:52	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	38 862	0	5	18	27	50	4.6/5.0
39/58	2026-04-01 11:14:12	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	38 096	0	6	18	27	51	4.6/5.0
38/58	2026-03-31 13:19:30	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	38 026	0	6	18	27	51	4.6/5.0
37/58	2026-03-31 07:11:59	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	38 039	0	6	18	27	51	4.6/5.0
36/58	2026-03-31 07:09:57	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	38 039	0	6	18	27	51	4.6/5.0
35/58	2026-03-30 12:00:35	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 337	0	6	18	24	48	4.6/5.0
34/58	2026-03-28 15:06:19	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 292	0	6	18	24	48	4.6/5.0
33/58	2026-03-28	completed	Config files,	37 292	0	6	18	24	48	4.6/5.0

	Date and Time	Status	Languages	Lines of Code	Number of Vulnerabilities					Score
					Critical	Medium	Low	Info	Total	
	14:43:49		Python, JavaScript, PL/SQL, T-SQL, HTML5							
32/58	2026-03-28 13:41:20	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	36 383	0	6	18	24	48	4.6/5.0
31/58	2026-03-27 12:59:46	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	36 185	0	6	18	24	48	4.6/5.0
30/58	2026-03-25 05:05:07	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	36 185	0	6	18	24	48	4.6/5.0
29/58	2026-03-21 14:33:54	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	36 137	0	6	18	24	48	4.6/5.0
28/58	2026-03-12 12:08:11	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 659	0	5	17	22	44	4.6/5.0
27/58	2026-03-12 11:12:50	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 661	0	5	17	22	44	4.6/5.0
26/58	2026-03-12 09:51:11	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 647	0	5	17	22	44	4.6/5.0
25/58	2026-03-12 05:08:40	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 603	0	5	17	22	44	4.6/5.0

	Date and Time	Status	Languages	Lines of Code	Number of Vulnerabilities					Score
					Critical	Medium	Low	Info	Total	
24/58	2026-03-11 14:34:48	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 603	0	5	17	22	44	4.6/5.0
23/58	2026-03-11 12:40:39	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 605	0	5	17	22	44	4.6/5.0
22/58	2026-03-11 10:19:20	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 605	0	5	17	22	44	4.6/5.0
21/58	2026-03-11 10:17:47	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 605	0	5	17	22	44	4.6/5.0
20/58	2026-03-11 06:46:08	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 605	0	5	17	22	44	4.6/5.0
19/58	2026-03-09 17:47:35	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 604	0	5	17	22	44	4.6/5.0
18/58	2026-03-05 06:32:44	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 175	0	1	12	21	34	4.8/5.0
17/58	2026-03-05 02:54:25	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 179	0	1	11	21	33	4.8/5.0
16/58	2026-03-03 12:15:09	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 177	0	1	11	21	33	4.8/5.0

	Date and Time	Status	Languages	Lines of Code	Number of Vulnerabilities					Score
					Critical	Medium	Low	Info	Total	
15/58	2026-03-03 11:17:09	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 174	0	1	11	21	33	4.8/5.0
14/58	2026-03-03 06:26:08	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 136	0	1	10	21	32	4.8/5.0
13/58	2026-03-03 05:51:38	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 304	3	9	12	21	45	3.0/5.0
12/58	2026-03-02 12:50:36	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 293	3	38	12	21	74	2.8/5.0
11/58	2026-03-02 11:36:05	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 261	3	38	12	21	74	2.8/5.0
10/58	2026-02-25 13:46:51	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 025	3	33	12	21	69	2.8/5.0
9/58	2026-02-25 05:21:50	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 022	3	32	12	21	68	2.8/5.0
8/58	2026-02-24 09:14:46	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 002	2	29	12	21	64	3.1/5.0
7/58	2026-02-23 14:39:13	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 011	2	29	12	21	64	3.1/5.0

	Date and Time	Status	Languages	Lines of Code	Number of Vulnerabilities					Score
					Critical	Medium	Low	Info	Total	
6/58	2026-02-20 07:24:02	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 011	2	29	12	21	64	3.1/5.0
5/58	2026-02-20 07:19:32	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	37 025	2	29	9	21	61	3.1/5.0
4/58	2026-02-19 15:14:09	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	34 021	0	27	10	21	58	4.4/5.0
3/58	2026-02-19 15:14:00	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	33 997	0	27	10	21	58	4.4/5.0
2/58	2026-02-19 11:51:59	completed	Config files, Python, JavaScript, PL/SQL, T-SQL, HTML5	33 797	0	27	10	21	58	4.4/5.0
1/58	2026-02-19 11:03:41	completed	Config files	4	0	0	0	0	0	5.0/5.0

02

Scan Information

58/58 2026-04-22 11:54:49
10.127506

Scan Statistics

Status
completed

Score
3.6/5.0

Duration
0:05:28

Lines of Code
45 587

Vulnerabilities

1	15	17	30	63
Critical	Medium	Low	Info	Total

Language	Status	Duration	Lines of Code	Number of Vulnerabilities				
				Critical	Medium	Low	Info	Total
Config files	completed	0:00:02	27 121	0	3	6	0	9
Python	completed	0:05:17	16 971	1	12	9	30	52
JavaScript	completed	0:00:05	809	0	0	2	0	2
PL/SQL	completed	0:00:01	289	0	0	0	0	0
T-SQL	completed	0:00:01	288	0	0	0	0	0
HTML5	completed	0:00:00	109	0	0	0	0	0

Language Statistics

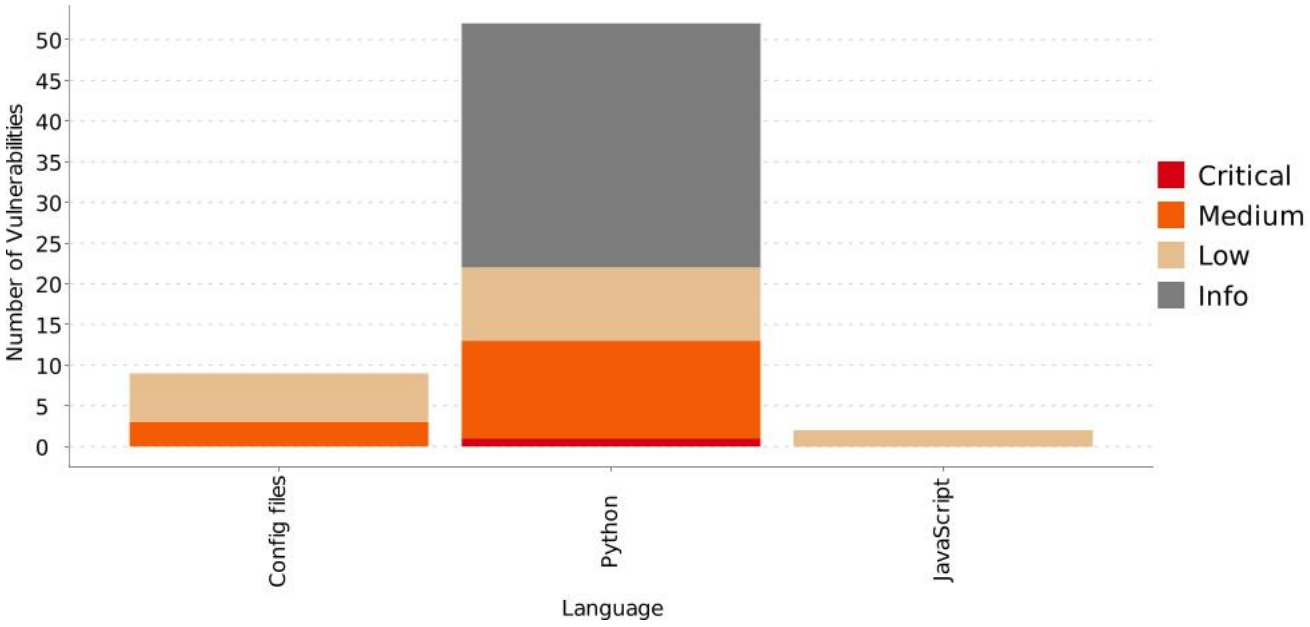
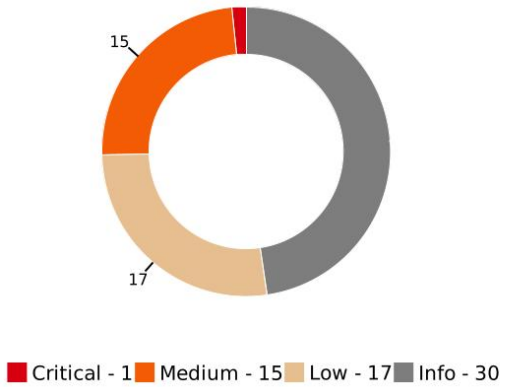


Diagram of identified vulnerabilities



Vulnerable Dependencies

Vulnerability List

Vulnerabilities are displayed accordingly to export settings: **16 selected**

Actual: **16 of 63**

Critical vulnerabilities		**1**
Hardcoded password	Python	1
src/api/mssql_source.py:219		Not processed
Medium-level vulnerabilities		**15**
HTTP usage	Config files	1
docker-compose.yml:48		Not processed
Password hardcoded in configuration file	Config files	2
docker-compose.yml:11		Not processed
docker-compose.yml:40		Not processed
Information leak	Python	6
src/db/database.py:186		Not processed
src/db/database.py:188		Not processed
src/db/database.py:252		Not processed
src/db/database.py:266		Not processed
src/db/database.py:301		Not processed
src/db/database.py:314		Not processed
Path manipulation	Python	5
scripts/check_vault.py:20		Not processed

* Rejected vulnerabilities are not taken into account

Medium-level vulnerabilities

Path manipulation	Python	
scripts/check_vault.py:25		Not processed
scripts/explore_remote_mssql.py:62		Not processed
scripts/explore_remote_mssql.py:66		Not processed
scripts/explore_remote_mssql.py:369		Not processed
Resource injection	Python	1
scripts/explore_remote_mssql.py:370		Not processed
Low-level vulnerabilities		0
Info-level vulnerabilities		0

Analysis Results

Hardcoded password (Python)

Description

Password is hardcoded. This may lead to an application data compromise.
Eliminating security risks related to hardcoded passwords is extremely difficult. These passwords are at least accessible to every developer of the application. Moreover, after the application is installed, removing password from its code is possible only via an update. Constant strings are easily extracted from the compiled application by decompilers. Therefore, an attacker does not necessarily need to have an access to the source code to know the parameters of the special account. If these parameters become known to an attacker, system administrators will be forced either to neglect the safety, or to restrict the access to the application.

Example

In the following example, a password is hardcoded:
password = "hardcoded"

Recommendations

- Store not passwords but values of cryptographically secure hash function from the password. Use specialized hash functions designed for this purpose (bcrypt, PBKDF2, scrypt). Use salt obtained from cryptographically secure pseudorandom number generator to resist attacks which use rainbow tables.
- If the hardcoded password is used for the initial authorization, provide the special authentication mode for this purpose in which the user is required to provide his/her own unique password.
- Store authentication information in an encrypted form in a separate configuration file or in a database. Secure the encryption key. If encryption is not possible, limit the access to the repository as much as possible.

Links

1. Use of hard-coded password

2. CWE-259: Use of Hard-coded Password
3. OWASP Top 10 2013-A5-Security Misconfiguration
4. OWASP Top 10 2013-A6-Sensitive Data Exposure
5. Handling passwords used for auth in source code - stackoverflow.com
6. How to securely hash passwords? - security.stackexchange.com
7. OWASP Top 10 2017 A2-Broken Authentication
8. OWASP Top 10 2017-A3-Sensitive Data Exposure
9. CWE-798: Use of Hard-coded Credentials
10. CWE CATEGORY: OWASP Top Ten 2017 Category A2 - Broken Authentication
11. CWE CATEGORY: OWASP Top Ten 2017 Category A6 - Security Misconfiguration

Vulnerability Entries

src/api/mssql_source.py:219

Level Critical

Status Not processed

```
216 # из connection string. Значение (сам пароль) приходит из Vault и в коде
217 # никогда не хранится. Переменная намеренно названа нейтрально, чтобы
218 # SAST-линтеры (DerScanner и пр.) не считали это hardcoded credential.

219 _ALIAS_SECRET_KEYS = {"pwd", "password"}

220 _ALIAS_TRUST_CERT = {"trustservercertificate"}
221 _ALIAS_ENCRYPT = {"encrypt"}
222 _ALIAS_TIMEOUT = {"connecttimeout", "connectiontimeout", "timeout"}
```

HTTP usage (Config files)

Description

Using HTTP rather than HTTPS allows “the man in the middle” attack. This can lead to a complete confidentiality loss of the transferred data.
Using HTTPS, which is based on HTTP and SSL / TLS, helps to protect the transferred data against unauthorized access and modification. It is recommended to use HTTPS for all cases of data transfer between the client and the server, in particular, for the login page and all pages that require authentication.

Example

In the following example, the application stores an address with HTTP protocol:
url = "http://example.com"

Recommendations

- Use only secure protocols (e.g., HTTPS) for the confidential data transfer between the client and the server.

Links

1. OWASP Top 10 2017-A3-Sensitive Data Exposure
2. Transport Layer Protection Cheat Sheet – OWASP
3. Web Security: Why You Should Always Use HTTPS – Mike Shema / Mashable
4. CWE-319: Cleartext Transmission of Sensitive Information
5. CWE CATEGORY: OWASP Top Ten 2017 Category A6 - Security Misconfiguration

Vulnerability Entries

docker-compose.yml:48

Level Medium

Status Not processed

```
45 #     condition: service_healthy
46 healthcheck:
47   # TCP-проба внутри контейнера: проверяем, что uvicorn слушает порт.
48   # Не используем http:// URL — во-первых, внутри контейнера до TLS мы
49   # всё равно не дотянемся (HTTPS делает ingress), во-вторых, для SAST
50   # TCP-проба выглядит однозначно безопасной и не цепляет "HTTP without TLS".
51   test:
```

Password hardcoded in configuration file (Config files)

Description

Saving a plaintext password inside the configuration file can lead to a system vulnerability.

Example

This is a code fragment in myApplication.properties where the password is set in an explicit form:

```
register.new = Register New User

username = USERNAME
password = PASSWORD
```

This is a code fragment in info.plist where the password is set in an explicit form:

```
<key>password</key>
<string>freerfe</string>
```

Recommendations

The password should never be a plaintext inside the configuration file. At best, the password must be entered by the system administrator when it is started. If such a method is not possible, then it is recommended that the password be artificially confused so that as much system resources as possible can be used to decrypt the password.

Links

1. Password Plaintext Storage-OWASP
2. CWE-798: Use of Hard-coded Credentials
3. CWE CATEGORY: OWASP Top Ten 2017 Category A2 - Broken Authentication
4. CWE CATEGORY: OWASP Top Ten 2017 Category A6 - Security Misconfiguration
5. CWE-256: Unprotected Storage of Credentials
6. CWE-260: Password in Configuration File

Vulnerability Entries

docker-compose.yml:11

Level Medium

Status Not processed

```
8 #   environment:
9 #     POSTGRES_DB: bank_statements
10 #     POSTGRES_USER: bank_user
11 #     POSTGRES_PASSWORD: "1234567890"
```

```
12 #   ports:
13 #     - "5432:5432"
14 #   volumes:
```

docker-compose.yml:40

Level Medium

Status Not processed

```
37  # - DB_PORT=5432
38  # - DB_NAME=bank_statements
39  # - DB_USER=bank_user
40  # - DB_PASSWORD=1234567890
41 restart: always
42 command: ["uvicorn", "src.api.app:app", "--host", "0.0.0.0", "--port", "8000"]
43 # depends_on:
```

Information leak (Python)

Description

System configuration information leak is possible. This can help an attacker to create a plan of an attack. Debug information and error messages can be written to the log, displayed to the console, or sent to the user depending on the system settings. In some cases, an attacker can make a conclusion about the system vulnerabilities from the error message. For example, a database error can indicate insecurity against SQL injection attacks. Information about the version of the operating system, server applications and system configurations can also be of value to the attacker.

Example

In the following example, the application turns on debug mode:
from django.conf import settings
settings.configure(DEBUG=True)

Recommendations

- Exclude detailed information about the system and its configuration from the error messages.

Links

1. OWASP Top 10 2013-A5-Security Misconfiguration
2. CWE-497: Exposure of System Data to an Unauthorized Control Sphere
3. OWASP Top 10 2017-A3-Sensitive Data Exposure
4. OWASP Top 10 2017-A6-Security Misconfiguration
5. CWE CATEGORY: OWASP Top Ten 2017 Category A5 - Broken Access Control
6. CWE CATEGORY: OWASP Top Ten 2017 Category A6 - Security Misconfiguration
7. CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
8. CWE-209: Generation of Error Message Containing Sensitive Information
9. CWE-489

Vulnerability Entries

src/db/database.py:186

Level Medium

Status Not processed

```
183    self.connection = psycopg2.connect(
184        **connect_kwargs
185    )
186    logger.info("Connected to %s@%s:%s", self.database, self.host, self.port)
187 except psycopg2.Error as e:
188    logger.exception("Connection failed: %s", e)
189    raise
```

src/db/database.py:188

Level Medium

Status Not processed

```
185         )
186         logger.info("Connected to %s@%s:%s", self.database, self.host, self.port)
187     except psycopg2.Error as e:
188         logger.exception("Connection failed: %s", e)
189         raise
190
191 def disconnect(self):
```

src/db/database.py:252

Level Medium

Status Not processed

```
249     return result[0] if result else None
250 except psycopg2.Error as e:
251     self.connection.rollback()
252     logger.exception("Insert error: %s", e)
253     raise
254 finally:
255     cursor.close()
```

src/db/database.py:266

Level Medium

Status Not processed

```
263     return cursor.rowcount
264 except psycopg2.Error as e:
265     self.connection.rollback()
266     logger.exception("Batch insert error: %s", e)
267     raise
268 finally:
```

```
269     cursor.close()
```

src/db/database.py:301

Level Medium

Status Not processed

```
298     return [r[0] for r in rows]
299 except psycopg2.Error as e:
300     self.connection.rollback()
301     logger.exception("Batch insert error: %s", e)
302     raise
303 finally:
304     cursor.close()
```

src/db/database.py:314

Level Medium

Status Not processed

```
311     self.connection.commit()
312 except psycopg2.Error as e:
313     self.connection.rollback()
314     logger.exception("Command error: %s", e)
315     raise
316 finally:
317     cursor.close()
```

Path manipulation (Python)

Description

Using data from an untrusted source when working with the file system may give an attacker access to important system files.
By manipulating variables that reference files with “dot-dot-slash (../)” sequences and its variations or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system including application source code or configuration and critical system files.

Example

In the following example, the application uses the value of the HTTP request parameter to specify the name of the file that is to be deleted. An attacker can set the string ../../tomcat/conf/server.xml as a parameter and thus delete the configuration file.

```
file_path = request.args.get("path")
os.remove(r"C:/" + file_path)
```

Recommendations

- Create a white list of acceptable names from which the user can choose. Do not use values entered by the user without validation.

Links

1. OWASP Top 10 2017-A1-Injection
2. OWASP Top 10 2017-A5-Broken Access Control
3. OWASP Top 10 2013-A4-Insecure Direct Object References
4. CWE-73: External Control of File Name or Path
5. Path Traversal - OWASP
6. CWE CATEGORY: OWASP Top Ten 2017 Category A1 - Injection
7. CWE-23
8. CWE-36
9. Restrict path access to prevent path traversal
10. A01:2021 - Broken Access Control
11. CWE-35: Path Traversal
12. A03:2021 - Injection

Vulnerability Entries

scripts/check_vault.py:20

Level Medium

Status Not processed

```
17 import sys
18
19 # project root
20 _root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
21 sys.path.insert(0, _root)
22 # optional: load .env from project root
23 try:
```

scripts/check_vault.py:25

Level Medium

Status Not processed

```
22 # optional: load .env from project root
23 try:
24     from dotenv import load_dotenv
25     load_dotenv(os.path.join(_root, ".env"))
26 except ImportError:
27     pass
28
```

scripts/explore_remote_mssql.py:62

Level Medium

Status Not processed

```
59
60
61 # Поднимаем .env из корня bankparserapi (если есть), чтобы не плодить экспорты.
62 _root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
```

```
63 sys.path.insert(0, _root)
64 try:
65     from dotenv import load_dotenv  # type: ignore
```

scripts/explore_remote_mssql.py:66

Level Medium

Status Not processed

```
63 sys.path.insert(0, _root)
64 try:
65     from dotenv import load_dotenv  # type: ignore
66     load_dotenv(os.path.join(_root, ".env"))
67 except ImportError:
68     pass
69
```

scripts/explore_remote_mssql.py:369

Level Medium

Status Not processed

```
366 ext = (extension or "").lstrip(".")
367 if ext and not safe_name.lower().endswith(f".{ext.lower()}"):
368     safe_name = f"{safe_name}.{ext}"
369 path = os.path.join(out_dir, f"{statement_id}__{safe_name}")
370 with open(path, "wb") as f:
371     f.write(bytes(data))
372 saved += 1
```

Resource injection (Python)

Description

An attacker can gain access to reading and changing protected system resources of the application has the ability to change the resource identifier.
Resource injection occurs when an atacker can specify identifier that will be used to access the system resource (for example, the port number to connect to the network resource). This allows him, in particular, to transfer valuable data to thied-party server.

Example

In the following example, the program executes a HTTP request in order to find out the price of tickets:

```
host_name = request.GET['host_example']
dbc = db.connect(host = host_name, port = 1235, dbname = ticket_data_base)
c = dbc.cursor()
###
result = c.execute('SELECT * FROM price_list')
###
```

If the request contains special characters, the attacker can receive confidential data.

Recommendations

- Create a whitelist of valid resource IDs and allow a user to select from this list and not to set his/her own value.
- If maintaining a whitelist is too difficult because of the large number of valid IDs, create a whitelist of characters allowed in identifiers. Blacklist in this case is ineffective, as it is likely to initially be incomplete, or sooner or later cease to be relevant.
- If nonetheless blacklist is chosen as a validation mechanism, make sure that it takes into account all the possible encodings and special character values (different for different operating systems). Changing the list should be simple when changing the requirements for validation.

Links

1. OWASP Top 10 2017-A1-Injection
2. OWASP Top 10 2013-A1-Injection
3. OWASP Top 10 2013-A4-Insecure Direct Object References
4. CWE-99: Improper Control of Resource Identifiers (‘Resource Injection’)
5. Dangerous Python Functions, Part 2
6. Create, use, and remove temporary files securely
7. CWE CATEGORY: OWASP Top Ten 2017 Category A1 - Injection
8. CWE-1030

Vulnerability Entries

scripts/explore_remote_mssql.py:370

Level Medium

Status Not processed

```
367 if ext and not safe_name.lower().endswith(f".{ext.lower()}"):
368     safe_name = f"{safe_name}.{ext}"
369 path = os.path.join(out_dir, f"{statement_id}__{safe_name}")
370 with open(path, "wb") as f:
371     f.write(bytes(data))
372 saved += 1
373 print(f"   ✓ {path}  ({len(bytes(data)):,} bytes)")
```

WAF Setup Instructions

HTTP usage

Description

Using HTTP rather than HTTPS allows “the man in the middle” attack. This can lead to a complete confidentiality loss of the transferred data.
Using HTTPS, which is based on HTTP and SSL / TLS, helps to protect the transferred data against unauthorized access and modification. It is recommended to use HTTPS for all cases of data transfer between the client and the server, in particular, for the login page and all pages that require authentication.

Vulnerability Entries

1. docker-compose.yml:48

Information leak

Description

System configuration information leak is possible. This can help an attacker to create a plan of an attack. Debug information and error messages can be written to the log, displayed to the console, or sent to the user depending on the system settings. In some cases, an attacker can make a conclusion about the system vulnerabilities from the error message. For example, a database error can indicate insecurity against SQL injection attacks. Information about the version of the operating system, server applications and system configurations can also be of value to the attacker.

Vulnerability Entries

1. src/db/database.py:186
2. src/db/database.py:188
3. src/db/database.py:252
4. src/db/database.py:266
5. src/db/database.py:301
6. src/db/database.py:314

Path manipulation

Description

Using data from an untrusted source when working with the file system may give an attacker access to important system files.
By manipulating variables that reference files with “dot-dot-slash (../)” sequences and its variations or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system including application source code or configuration and critical system files.

Vulnerability Entries

1. scripts/check_vault.py:20
2. scripts/check_vault.py:25
3. scripts/explore_remote_mssql.py:62
4. scripts/explore_remote_mssql.py:66
5. scripts/explore_remote_mssql.py:369

Resource injection

Description

An attacker can gain access to reading and changing protected system resources of the application has the ability to change the resource identifier.
Resource injection occurs when an atacker can specify identifier that will be used to access the system resource (for example, the port number to connect to the network resource). This allows him, in particular, to transfer valuable data to thied-party server.

Vulnerability Entries

1. scripts/explore_remote_mssql.py:370

Scan Settings

58/58 2026-04-22 11:54:49

Select files for Analysis

**/*

Branch in Repository

master

Languages

- [ ] ABAP
- [ ] Apex
- [x] C#
- [ ] C/C++
- [ ] COBOL
- [x] Config files
- [x] Dart
- [x] Go
- [ ] Groovy
- [x] HTML5
- [x] Java, Scala, Kotlin
- [x] JavaScript, TypeScript
- [ ] LotusScript
- [ ] Objective-C
- [ ] Delphi, Pascal
- [x] PHP
- [x] PL/SQL, T-SQL
- [x] Python
- [ ] Perl
- [x] Ruby
- [x] Rust
- [x] Solidity
- [x] Swift
- [ ] TSX
- [ ] VB.NET, VBA, VBScript, VB6
- [ ] Vyper
- [ ] 1C

Java/Scala/Kotlin Specific Settings

- (●) Prebuilt project with .class files
- ( ) Source code (will be built by DerScanner)
- ( ) Source code (no build needed, only for Java)
- ( ) Source code (build with own tools)

JavaScript Specific Settings

- [ ] Analyze standard libraries

General Analysis Settings

- [ ] Analyze libraries and nested archives
- [ ] Use extra rules
- [ ] Incremental analysis
- [x] Preprocessing

Source Code Charset	UTF-8
Filename Charset	UTF-8

Rule Sets —

03 Export Settings

Project Information

Project Statistics

- [x] Dynamics by rating
- [x] Dynamics by vulnerabilities

Scan History

- [ ] Do not export scan history
- [x] Export entire scan history
- [ ] Export the latest scans ...

Vulnerability Classification

By severity

Scan Information

- [x] Detected vulnerabilities chart
- [x] Vulnerable Dependencies
- [x] Statistics on languages
- [ ] Analyzed files statistics
- [x] Error information
- [x] Scan settings
- [x] Version Properties

Vulnerability Filter

Severity

- [x] Critical
- [x] Medium
- [ ] Low
- [ ] Info

Vulnerable Dependencies

- [x] In standard libraries

- [x] In .class files that cannot be decompiled
- [x] Vulnerabilities without WAF configuration guide

Languages

- [x] ABAP
- [x] Android
- [x] Apex
- [x] C#
- [x] C/C++
- [x] COBOL
- [x] Config files
- [x] Dart
- [x] Delphi
- [x] Go
- [x] Groovy
- [x] HTML5
- [x] Java
- [x] JavaScript
- [x] Kotlin
- [x] LotusScript
- [x] Objective-C
- [x] Pascal
- [x] PHP
- [x] PL/SQL
- [x] Python
- [x] Perl
- [x] Ruby
- [x] Rust
- [x] Scala
- [x] Solidity
- [x] Swift
- [x] T-SQL
- [x] TSX
- [x] TypeScript
- [x] VB.NET
- [x] VBA
- [x] VBScript
- [x] Visual Basic 6
- [x] Vyper
- [x] 1C

Vulnerability List

Vulnerability Statuses

- [x] Not processed
- [x] Confirmed
- [ ] Rejected

List of Vulnerability Entries

- ○ Do not export
- ● Export all entries
- ○ Export no more than entries ...

Analysis Results

Vulnerability Statuses

- [x] Not processed

- [x] Confirmed
- [ ] Rejected

Vulnerability Entries

- ( ) Do not export
- (•) Export all entries
- ( ) Export no more than entries ...

Source Code

- ( ) Do not export source code
- ( ) Export entire vulnerable source code file
- (•) Export context in the number of lines of code 3

Trace

- ( ) Do not export trace items
- ( ) Export all items
- (•) Export only the first and last items

Additional information

- [x] Vulnerability comment
- [x] Vulnerability actions

WAF Setup Instructions

Guide for Vulnerability Statuses

- [x] Not processed
- [x] Confirmed
- [ ] Rejected

Guide for WAF

- [x] Imperva SecureSphere
- [x] ModSecurity
- [x] F5

General Report Settings

- [x] Table of contents
- [x] Report export settings
- [x] Display vulnerability statuses
- [x] Display results correlation tags